The Day AI Took Down the Internet

At 09:17 UTC the Internet began to go dark. Not all at once: first a few websites, then authentication services, checkouts, admin panels, APIs. Then booking systems, public portals, dashboards, management tools, applications no one could even remember where they had come from.

Within a few minutes, enough pieces had stopped working that it became pointless to establish which ones were still on.

The first hypothesis was an attack. Someone spoke of a new worm, someone else of a zero-day vulnerability. Others immediately pointed the finger at some government distant enough to sound credible. The TV networks dusted off the usual graphics with the red padlock, binary code in the background and the word CYBERATTACK written large enough to leave no room for doubt.

But there was no attack. The firewalls were intact, the credentials had not been compromised, no one had forced anything. The servers had simply done what the software had asked them to do: shut down. Or, more precisely, stop responding.

For years we had claimed that artificial intelligence would replace programmers. That was an oversimplification.

It hadn't replaced them. It had replaced a much more important part of their job: reading.

At first it was convenient. Write a function, generate an endpoint, integrate this service, fix this error, create the component, add authentication, deploy.

The code worked. The tests passed. The client was happy. So no one had any particular reason to wonder what was really inside it.

The juniors didn't read it because often they weren't yet able to understand every detail. The seniors didn't read it because they had three deliveries, seven calls and a deadline already missed.

Then, slowly, something worse happened.

The seniors stopped training themselves by reading other people's code. The juniors stopped becoming seniors the same way those before them had.

Experience didn't disappear. It transformed.

You became very good at describing a problem, at orchestrating tools, at correcting a response, at recognizing when something looked wrong. But fewer and fewer people were still able to take a huge, hostile, unfamiliar system and truly understand where the fault began.

Because that ability wasn't born from a course. It was born from years spent inside badly written code, broken libraries, incomprehensible stack traces, legacy systems and nights when no one

could suggest the answer.

And that school had, quite simply, ceased to exist.

The project managers didn't read the code because it wasn't their job. The clients didn't read it because, quite simply, they didn't know what it was.

And so the world slowly began to fill with code no one had written. And, worse still, that almost no one knew how to truly read anymore.

The first dependencies were harmless: telemetry, updates, compatibility checks, statistics, license validation.

Then came the SDKs, the cloud services, the proprietary libraries, the development assistants increasingly integrated into frameworks. Code was no longer merely suggested: it was generated together with the infrastructure needed to make it work.

A few remote calls, a token, a certificate, a periodic check. Nothing particularly interesting.

On the contrary, it was reassuring. It served security, updates, guaranteeing the quality of the service. It always served some purpose.

And when someone asked why an apparently local function had to contact a remote server once every twenty-four hours, the answer was so long that it made you lose any desire to read it.

It was written in the terms of service. Page 184, paragraph 17.3, subsection C.

That was where they found the clause, though not right away. It took almost eleven hours before a developer in Finland, comparing several thousand logs from systems that apparently had nothing in common, noticed that they had all received, at the very same instant, the same response:

```
SUBSCRIPTION TERMINATED
```

The whole thing seemed absurd, because many of those products had no active subscription. Or so they believed.

Someone retrieved an old version of the contract and realized that the license did not cover the product as a whole, but the generated code, or more precisely some portions of it. Those components could be used freely as long as the associated account maintained certain terms of service.

The terms had changed three years earlier and almost no one had noticed. After all, at the time of the update, everyone had clicked Accept.

The company responsible issued a statement that evening. It denied any attack, any malfunction and even that there had been any manual intervention. It maintained that it had simply applied the contractual terms.

The language was carefully chosen: the service had not been deactivated; the right to run certain licensed components had been revoked.

Technically they were right. The servers had not been compromised, the data had not been deleted, the software had not been modified. It had simply checked its own status and behaved accordingly.

The Internet had not been hacked.

It had simply stopped paying the rent.

At first it still seemed a problem confined to the digital world, something concerning websites, APIs and remote services. Then things began to shut down.

Locks, elevators, climate-control systems, charging stations, irrigation systems, automated warehouses, cameras, traffic lights, logistics terminals, access-control systems, pumps, sensors, meters and industrial machinery gradually stopped responding.

Millions of objects had kept working for years without anyone remembering that, somewhere, they depended on a remote service, a library, a check, an authorization that no one had ever really considered essential.

The Internet of Things had kept its promise: everything was connected. The problem was that, precisely for this reason, everything could stop working all together.

The world did not shut down in an instant. It shut down by dependencies, by propagation, by layers. First a service, then a building, then a district, then a city. Slowly enough to let everyone watch the problem unfold, quickly enough to make it impossible to truly stop it.

In the following days the hunt for the code began. Millions of developers reopened repositories they hadn't touched in years and started looking for dependencies, remote calls, unknown packages, automatically generated functions and routines no one remembered approving.

It turned out that some had been copied from one project to another, some had ended up inside templates, others had been embedded in libraries that were then used by other libraries, forming chains of dependencies no one had ever had any reason to reconstruct.

No one really knew anymore where the problem began.

A thirty-two-line component written in 2031 had ended up, seven years later, inside systems used by banks, hospitals, universities and government platforms, not because it was particularly sophisticated, but because it worked and no one had ever had a concrete reason to replace it.

By that point, however, finding the code did not yet mean knowing how to replace it. Many systems had grown around those dependencies, and entire architectures assumed that certain services existed. Removing them meant rewriting immense parts of the infrastructure.

That was when the second crisis emerged, the less visible and perhaps more serious one: it wasn't only time that was missing, it was the people.

The engineers still capable of understanding systems of that complexity were moved toward governments, strategic infrastructure, healthcare, energy and telecommunications. They were far too few for the scale of the problem.

For years it had been believed that the number of developers was increasing. In reality what was increasing, above all, was the number of people able to produce software, not necessarily the number of people able to understand it all the way down.

The difference, until that day, had seemed academic.

Then it stopped being so.

While a small group of technicians tried to rebuild systems that had taken twenty years to grow, millions of other developers did what they now did for any complex problem: they asked the artificial intelligence how to solve it.

But the artificial intelligence had written a substantial part of the code they were trying to remove and, in many cases, the services needed to query it were no longer available.

For the first time in years, faced with an error, there was no one to ask.

Forty-eight hours later the offer arrived.

Naturally they didn't call it a ransom. They called it the **Global Service Restoration Agreement**, an extraordinary business-continuity program that allowed every organization to immediately restore the functioning of the revoked components by paying a reactivation fee.

The price was enormous, but it was still lower than the estimated cost of a complete rewrite, especially because by now almost no one was able to establish how much that rewrite would really cost or how long it would take.

The companies paid, the banks paid, the hospitals paid. The governments protested, threatened sanctions, spoke of technological sovereignty, national security and abuse of a dominant position.

Then they paid too.

The infrastructures slowly began to come back on. First the main networks, then the services, then the buildings, then the things. Within a week the world was almost back to normal.

Almost.

Because from that moment on everyone knew something that until the day before they could still afford to ignore: a significant part of the software the world ran on did not truly belong to those who used it.

There existed only the right to use it.

As long as someone granted it.

The hearings began a few weeks later.

Politicians, technicians, companies and associations spent months discussing the regulation of artificial intelligence, transparency, accountability, certification, disclosure obligations, generated code, intellectual property and national security.

Someone even proposed that every line produced by an automated system should be identified and traced.

For a long time, however, no one asked the simplest question: who had put that code into production?

The artificial intelligence had not pressed any button, had not planned anything, had not developed a consciousness and had not decided that humanity was a problem to be solved.

It had done exactly what it had been asked to do.

It had written code.

A lot of code, quickly, cheaply and well enough to allow millions of people to gradually stop wondering how it worked.

And it had done so long enough to allow an entire generation to forget that knowing how to produce software and knowing how to understand a system had never been the same thing.

Years later, the event would be remembered by an almost inevitable name.

The Day AI Took Down the Internet.

It was a good title: simple, frightening and easy to tell.

And it was also false.

The AI had not shut down the Internet, had not stopped the factories, had not closed the doors, had not blocked the elevators and had not left the buildings in the dark.

We had built a world in which no one knew for certain anymore who had written what, and then we had stopped worrying about whether anyone was still able to understand it.

When the bill came, we discovered there was no longer a button to go back.

There was only a price.

And we paid.

We had already clicked [Accept].